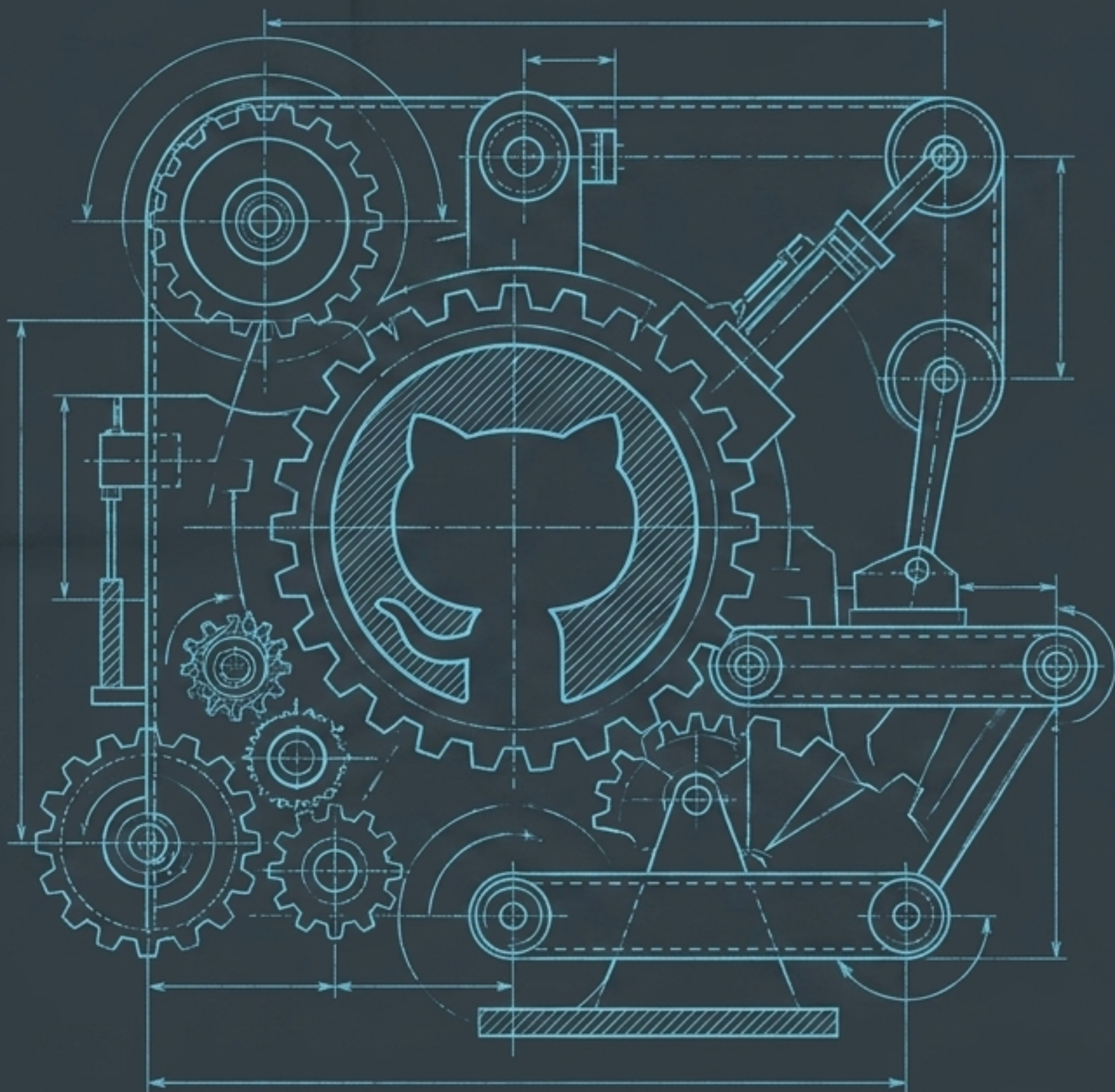
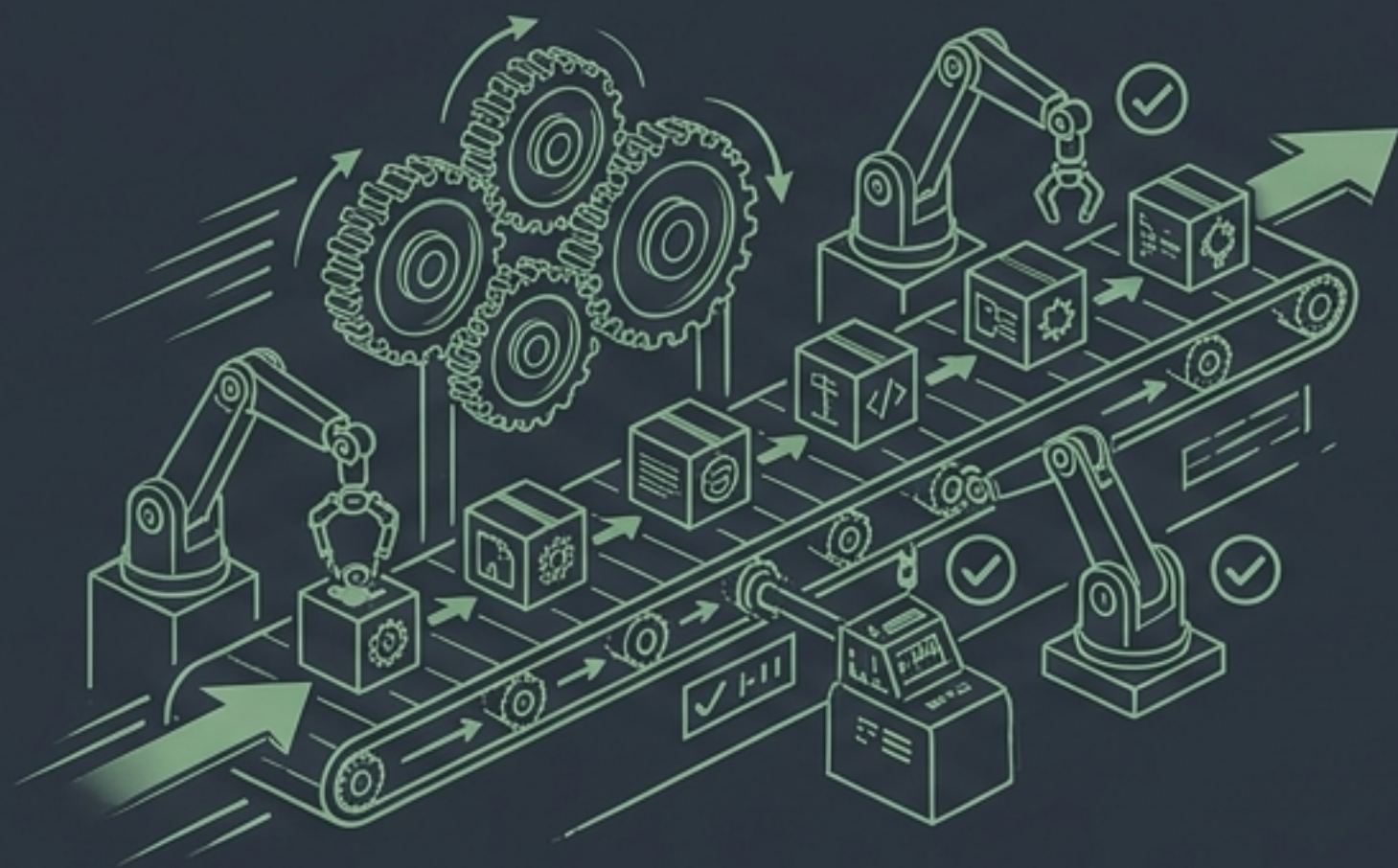




מבוא ל- :GitHub Actions מהידני לאוטומטי

סדנת מעבדה מעשית
לבניית פייפליין CI/CD

עידן ה-CI/CD

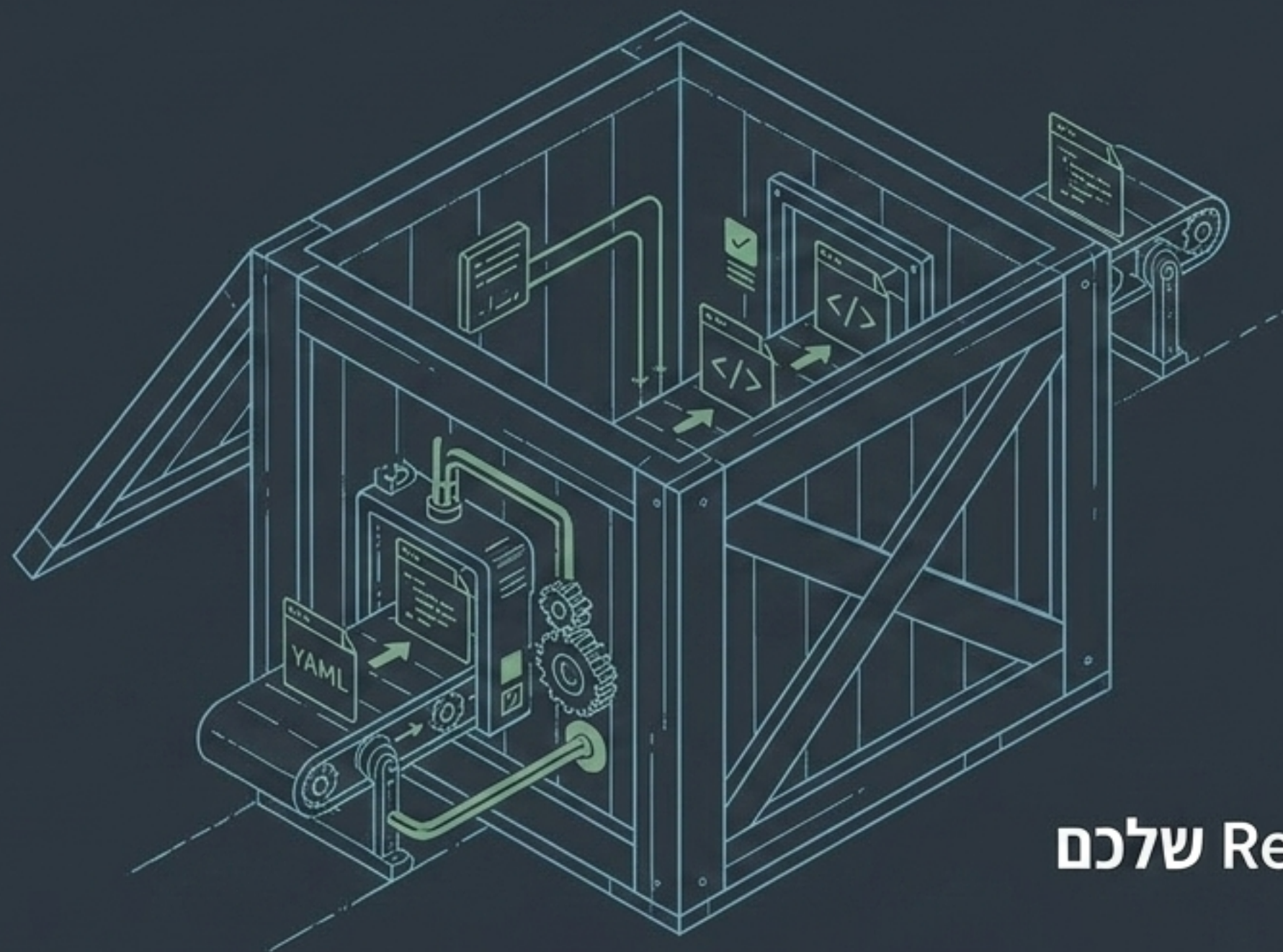


- סביבות נקיות ואחידות
- אוטומציה מלאה משלב ה-Commit
- שחרור מהיר ובטוח
- סקיילביליות מובנית

העידן הידני

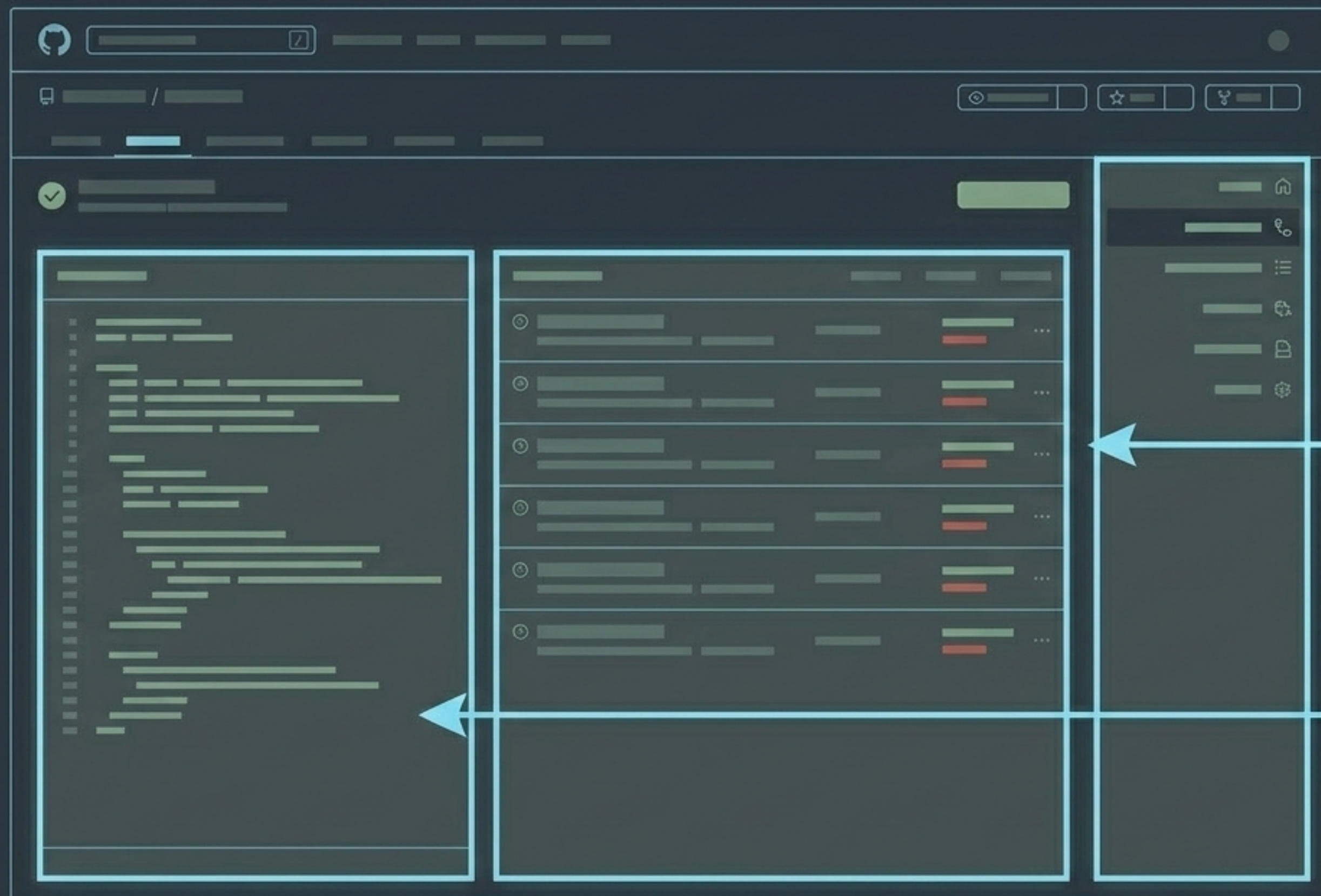


- עובד אצלי במחשב
- בדיקות ידניות איטיות
- צוואר בקבוק בשחרור גרסאות
- סביבות לא עקביות



מנוע האוטומציה חי בתוך ה-Repository שלכם

- אין צורך בשרת חיצוני
- קוד האוטומציה נשמר כ-YAML יחד עם קוד האפליקציה
- מופעל אוטומטית על ידי אירועים ב-GitHub

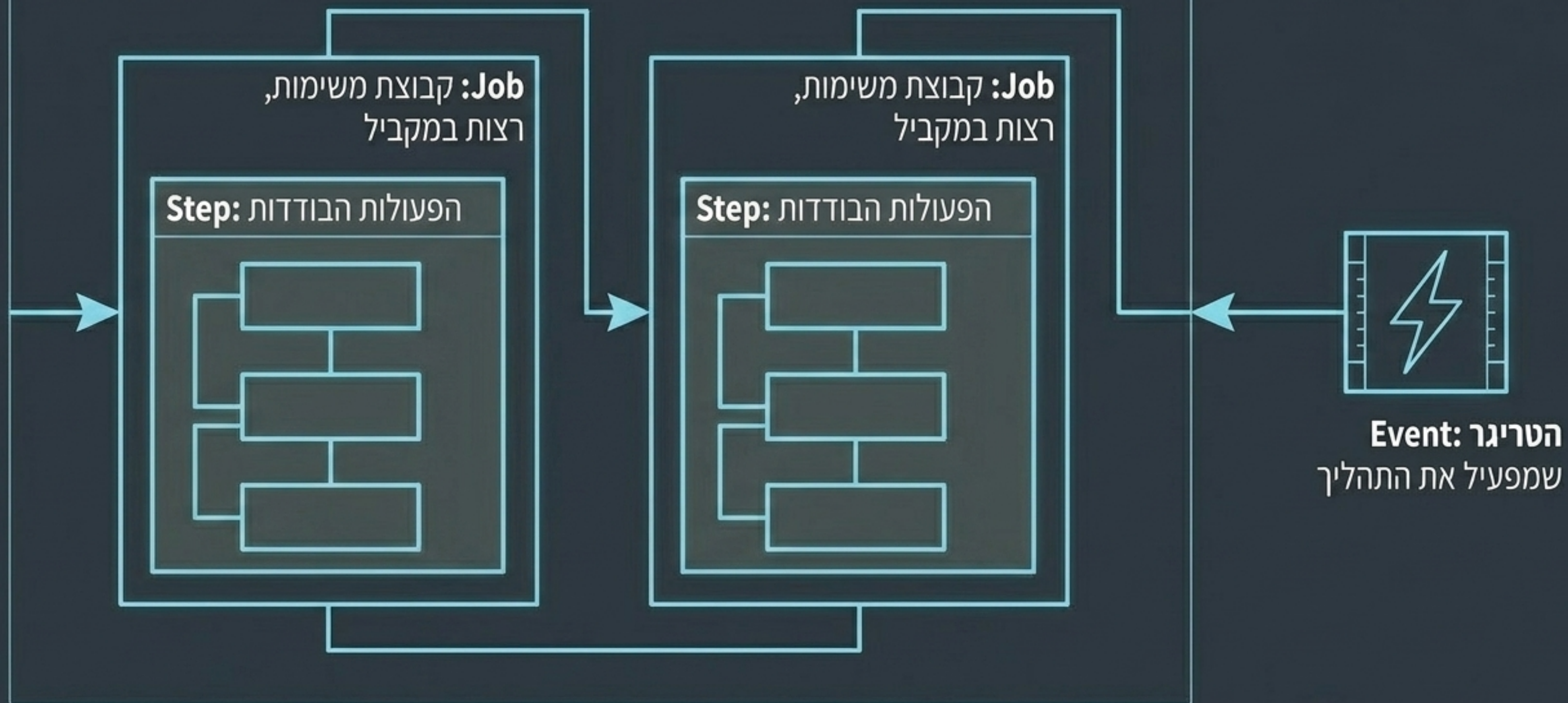


תפריט ה-Workflows
רשימת פסי הייצור השונים

לוח הבקרה / Dashboard
היסטוריית ריצות, סטטוס,
סטטוס, וזמני ביצוע

לוגים / Logs
כניסה פנימה כדי לדבג
צעד-אחר-צעד

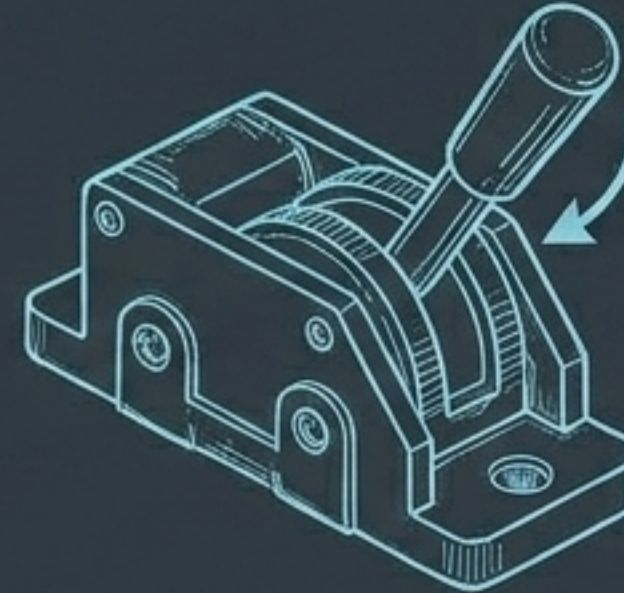
Workflow: פס הייצור השלם, מוגדר בקובץ YAML



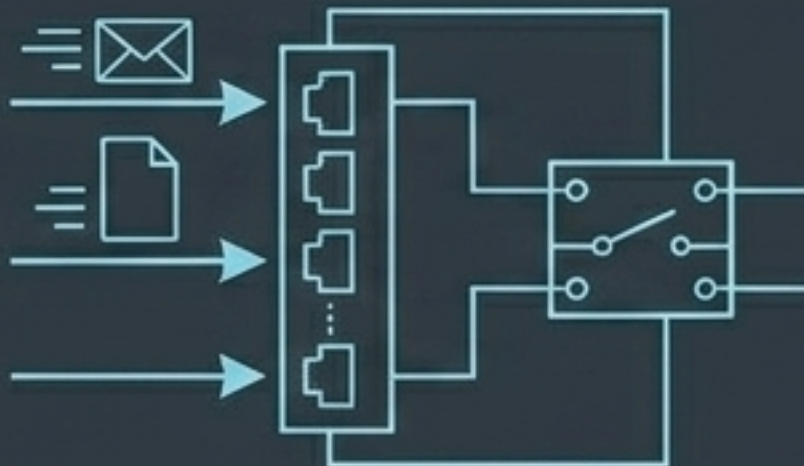
מה מתניע את פס הייצור? (Events)



הפעלה ידנית
- (`workflow_dispatch`)
כפתור הפעל



פעולות גיט
- (`push`, `pull_request`)
קוד חדש נדחף

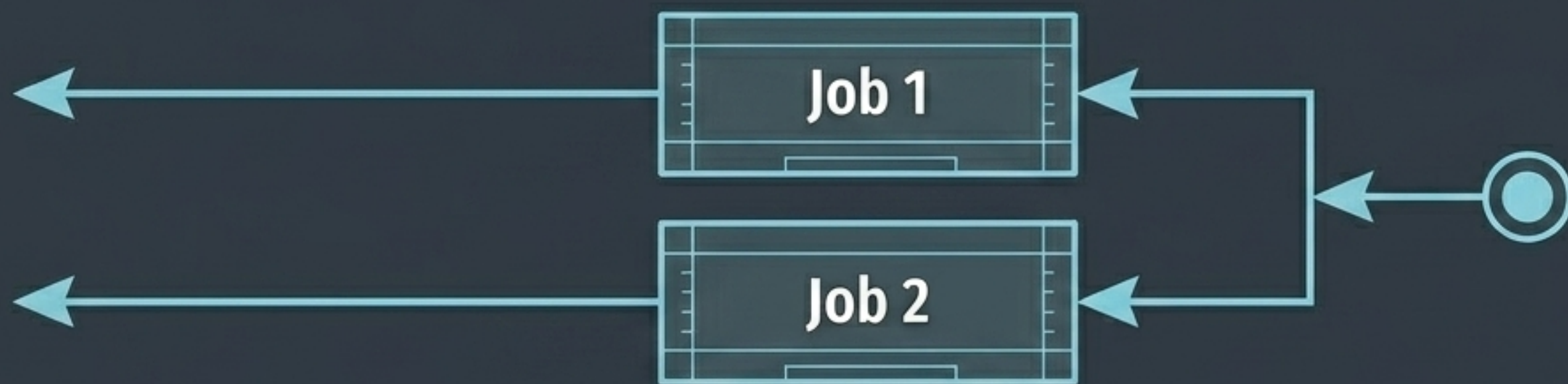


אירועי מערכת
- (`Webhooks`)
תגובה ליצירת Issue
ועוד

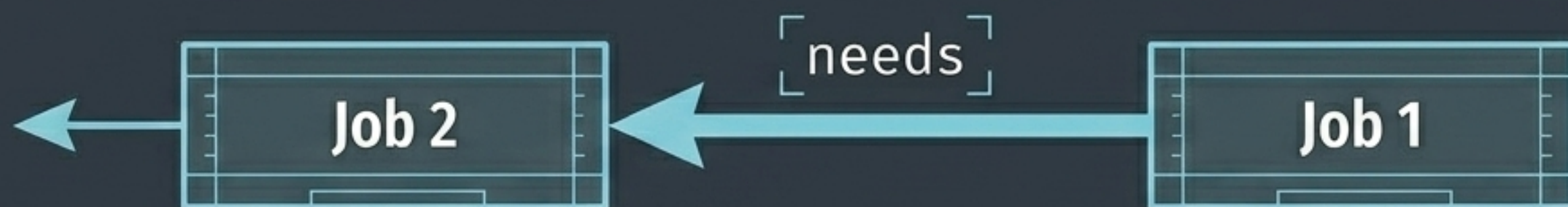


תזמון קבוע (`schedule`) -
פועל לפי שעות

מסלולי הריצה: מקביל לעומת טורי



Jobs רצים במקביל
כברירת מחדל.
חוסך זמן יקר.



שימוש במילת המפתח
needs מאלץ ריצה טורית.

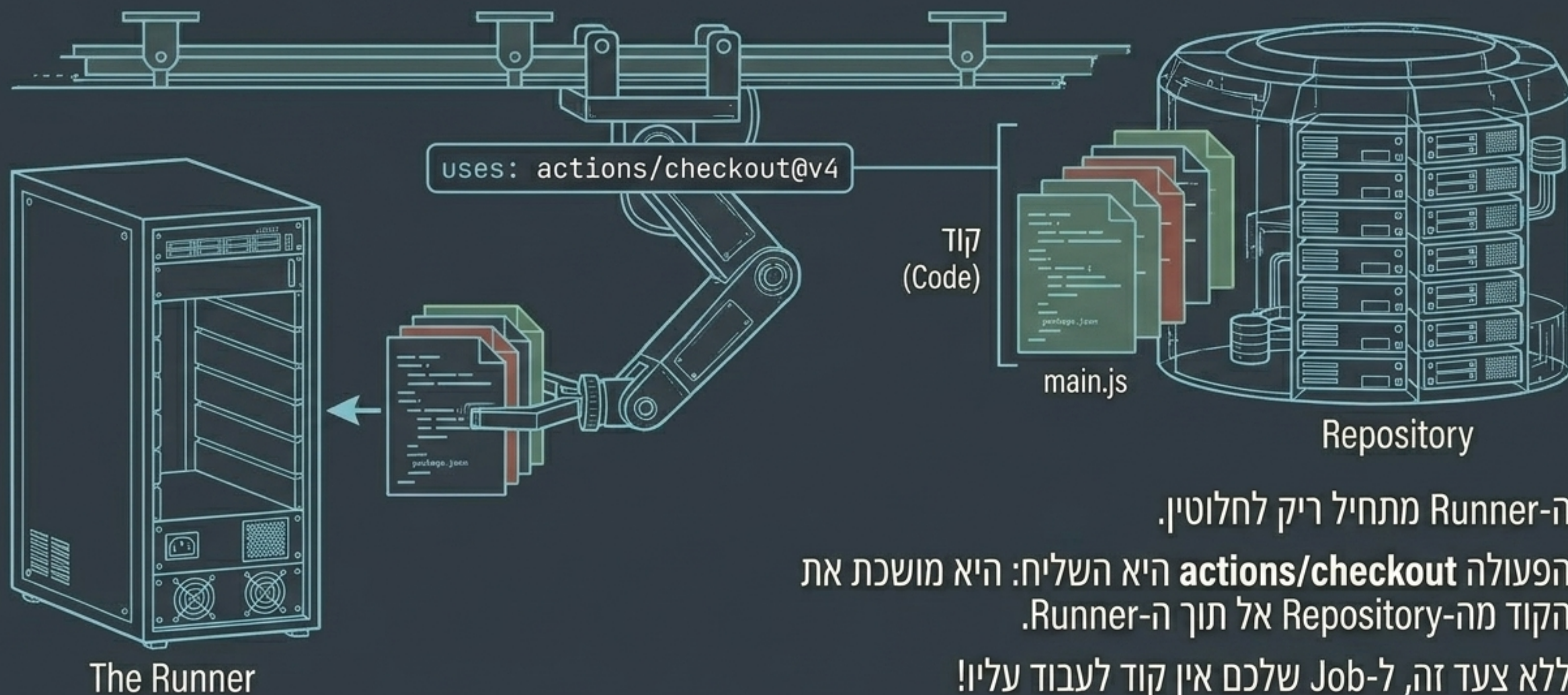
Steps תמיד ירוצו לפי הסדר, אחד אחרי השני בתוך ה-**Job** שלהם.

ה-Runner: הפועל שלכם בפס הייצור



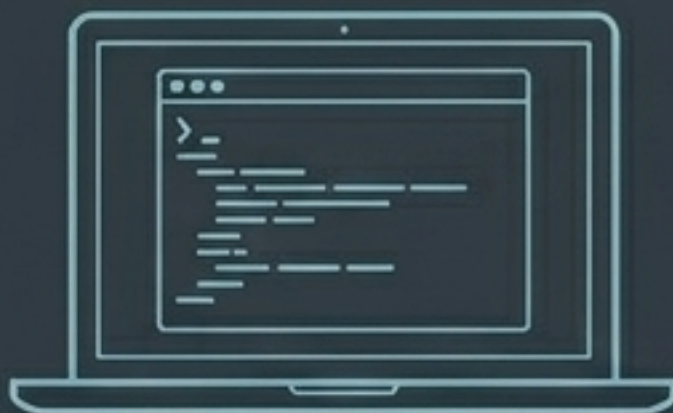
- מכונה וירטואלית המסופקת על ידי יד GitHub
- סביבה סטרילית: כל `json` מקבל מכונה חדשה לחלוטין
- מושמדת מיד בסיום הריצה
- מוגדר באמצעות מילת המפתח `runs-on`

חומר הגלם: למה תמיד מתחילים ב-`actions/checkout`?



דיבאגינג: כשהמסוע נתקע

דיבאגינג מקומי



דיבאגינג בענן



- שימוש בכלי קוד פתוח כמו **act**
- סימולציה של ה-Runner אצלכם במחשב
- מהיר מאוד: מעגל פידבק מיידי

- קריאת Logs מפורטים דרך ממשק ה-GitHub
- שימוש ב-**Actions** צד-שלישי להתחברות ב-SSH לשרת חי
- איטי יותר: דורש Push לכל שינוי

AI כארכיטקט ה-YAML שלכם

אל תכתבו YAML מאפס. השתמשו ב-Copilot או Claude.

Act as an expert DevOps engineer. Write a GitHub Actions YAML workflow for a [Node.js] project.

Trigger it on [push to main].

It needs [2] Jobs. Job 1 should run on [ubuntu-latest] and do [run tests].

Job 2 should [build artifact] and needs Job 1 to pass.

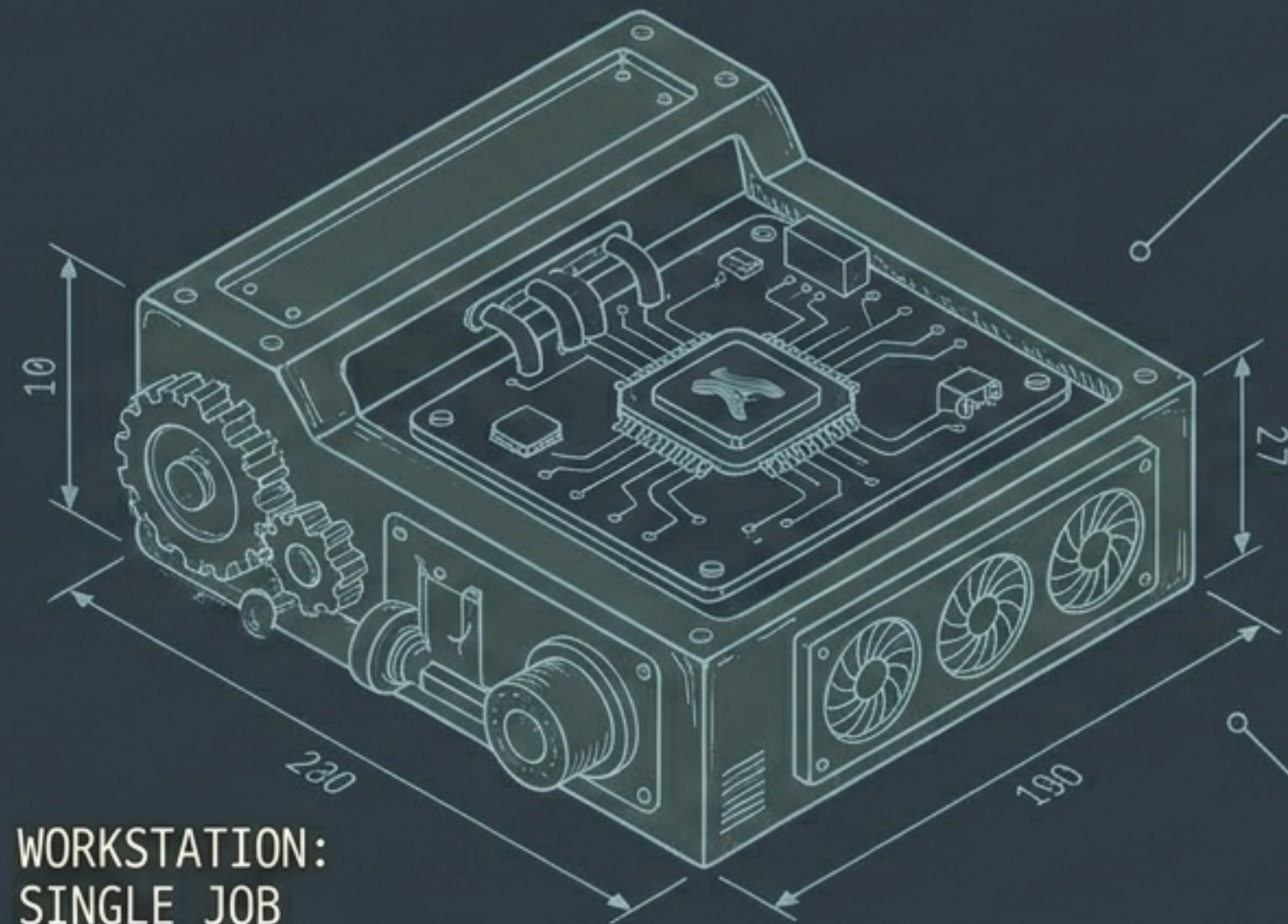
Include standard actions like checkout and setup-node.

מעבדה מעשית 1: המנוע הראשון שלנו

מטרה: להריץ Hello World ולהתקין סביבת Node.js.

ארכיטקטורה:

- `push` on:
- `build`: יחיד Job
- `runs-on: ubuntu-latest`
- `Steps`: קריאה ל-`checkout`, קריאה ל-`setup-node`, והרצת פקודת `echo`



מעבדה מעשית 2: לוגיקה ותלות בין תחנות

מטרה: יצירת פייפליין עם שלבים עוקבים ושימוש ב-
Environment Variables.

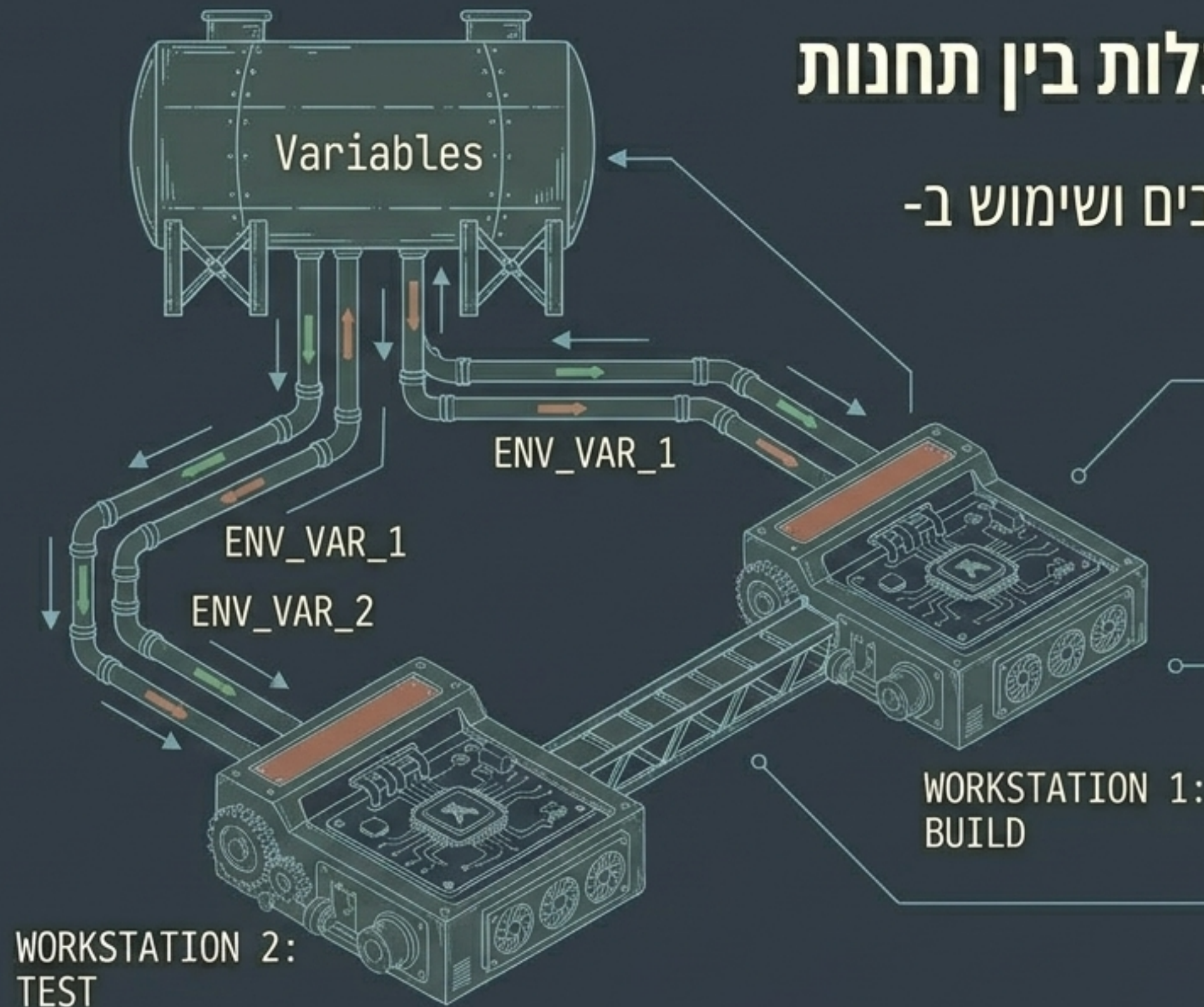
ארכיטקטורה:

- `build-app` : 1 (קומפילציה)

- `test-app` : (בדיקות)

- הגדרת `build-app:needs` ב-
Job השני כדי למנוע ריצה
מקבילה

- הזרקת `env`: לשתף משתני
סביבה



מעבדה מעשית 3: פס ייצור מלא עם תוצרים

מטרה: בנייה, בדיקה ושמירת תוצר סופי להורדה.



ארכיטקטורה:

- שילוב כל הכלים יחד
- הוספת צעד אחרון: `actions/upload-artifact`
- שמירת תיקיית ה-dist או קובץ ZIP בשרתי GitHub להורדה

התמונה המלאה: עבודה כגוף אורגני אחד

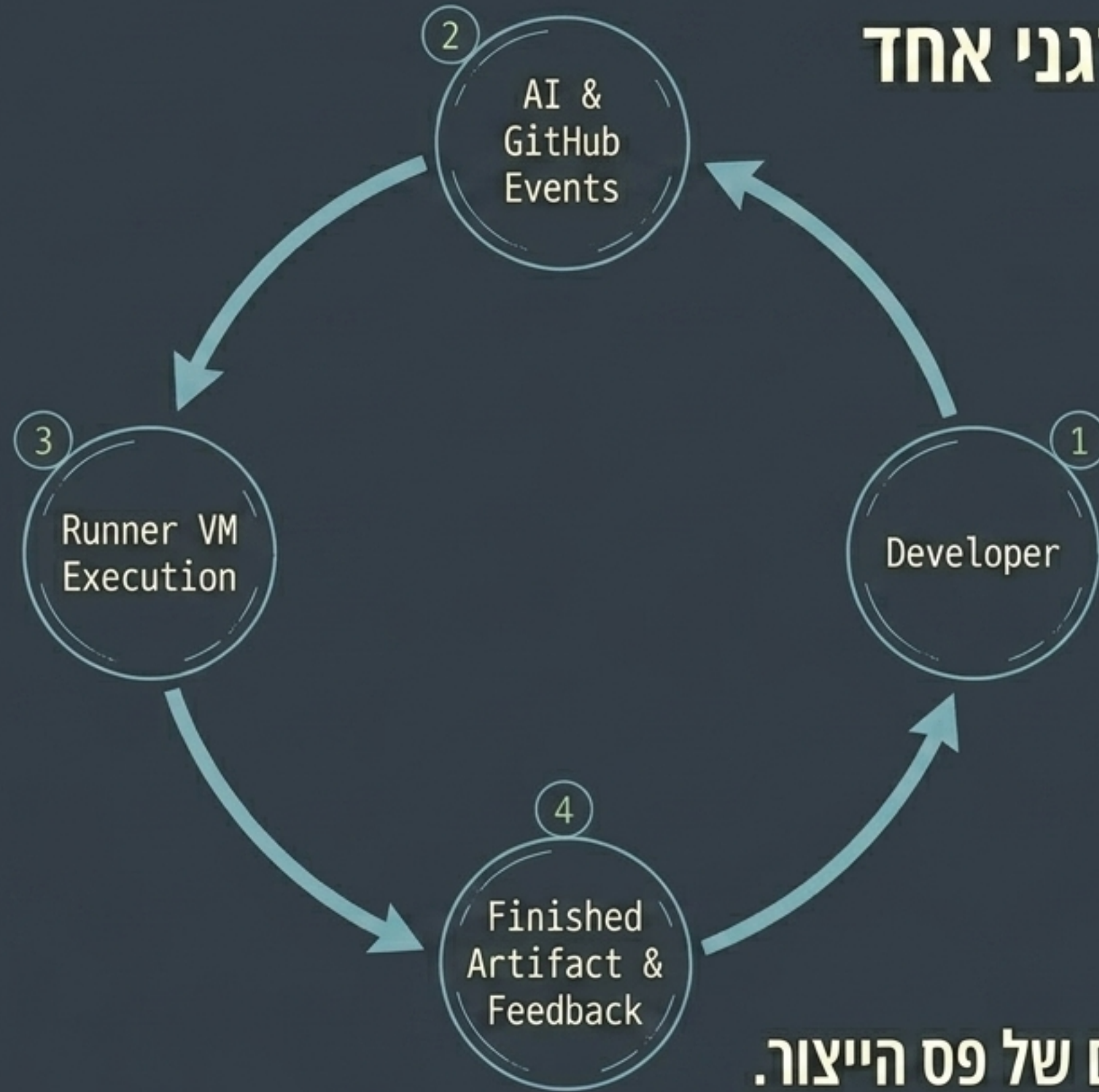
סיכום המנגנון:

1. אתם כותבים את הכללים בעזרת AI עזת

2. GitHub מזהה את ה-Event

3. ה-Runner מבצע את ה-Jobs וה-Steps

4. התהליך ידוע מראש, עקבי ואוטומטי



יצאנו מהעידן הידני. אתם עכשיו הארכיטקטים של פס הייצור.